

PROJECT REPORT

Customer Churn Prediction & Analytics Dashboard

An End-to-End Data Analytics Project — Python, SQL, Machine Learning & Power BI

SUBMITTED BY ANS Pranav Chaitanya

TOOLS & TECHNOLOGIES Python, MySQL, Power BI, Scikit-learn

DATASET Telco Customer Churn (Kaggle) — 7,043 records

Table of Contents

1. Introduction.....	3
2. Objectives.....	4
3. Tools & Technologies	5
4. Dataset Description.....	6
5. Project Plan.....	7
6. Methodology & Implementation.....	8
6.1 Step 1 — Data Loading, Cleaning & EDA (Python).....	8
6.2 Step 2 — SQL Database Setup & Churn Queries.....	12
6.3 Step 3 — Machine Learning: Churn Prediction Model.....	15
6.4 Step 4 — Power BI Interactive Dashboard	18
7. Results & Key Insights.....	20
8. Appendix A — Project Repository Structure	21
9. Conclusion.....	22
10. References.....	23

1. Introduction

Customer churn — the loss of paying customers to competitors or service discontinuation — is one of the most significant challenges facing subscription-based and telecom businesses. Even a small reduction in churn can have a substantial impact on long-term revenue, since acquiring a new customer is typically far more expensive than retaining an existing one.

This project presents an end-to-end customer churn analytics solution built on a real-world telecommunications dataset. It combines data cleaning and exploratory analysis in Python, relational data querying in SQL, predictive modelling using machine learning, and business-facing visualization through an interactive Power BI dashboard. Together, these components form a complete analytics pipeline — from raw data to actionable business insight — of the kind typically expected in a Data Analyst or Data Science role.

Project Summary

Field	Details
Project Title	Customer Churn Prediction & Analytics Dashboard
Prepared By	ANS Pranav Chaitanya
Tools Used	Python, SQL (MySQL), Power BI, Scikit-learn
Key Libraries	Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn
Dataset	Telco Customer Churn — Kaggle (7,043 records, 21 attributes)
ML Algorithm	Random Forest Classifier (benchmarked against Logistic Regression and Decision Tree)
Project Goal	Predict customer churn and visualize churn drivers for business decision-making

2. Objectives

The project was designed around the following objectives:

- Clean and explore a real-world telecom customer dataset to understand churn patterns.
- Use SQL to perform structured business queries on customer and revenue data.
- Build and evaluate a machine learning model capable of predicting customer churn.
- Identify the key factors that drive customers to churn.
- Design an interactive Power BI dashboard that communicates churn insights to non-technical stakeholders.
- Quantify the financial impact of churn on monthly and historical revenue.

3. Tools & Technologies

The project uses a standard data analytics technology stack, spanning data preparation, statistical analysis, machine learning, and business intelligence.

Category	Tools / Libraries
Programming Language	Python 3
Data Handling	Pandas, NumPy
Data Visualization	Matplotlib, Seaborn
Machine Learning	Scikit-learn (Logistic Regression, Decision Tree, Random Forest)
Database	MySQL
Business Intelligence	Microsoft Power BI (Power Query, DAX)
Version Control	Git & GitHub

4. Dataset Description

The project uses the publicly available Telco Customer Churn dataset from Kaggle, which contains 7,043 customer records across 21 attributes for a fictional telecommunications company.

- Demographic information: gender, SeniorCitizen, Partner, Dependents.
- Account information: tenure, Contract type, PaperlessBilling, PaymentMethod.
- Services subscribed: InternetService, PhoneService, and related add-ons.
- Billing information: MonthlyCharges, TotalCharges.
- Target variable: Churn (Yes / No) — whether the customer left the company.

5. Project Plan

The project was executed over four weeks, progressing from raw data to a finished business intelligence dashboard.

Step	Task	Tools	Timeline
1	Data Loading, Cleaning & Exploratory Data Analysis	Python, Pandas, Matplotlib, Seaborn	Week 1
2	SQL Analysis & KPI Queries	MySQL, SQL	Week 2
3	Machine Learning — Churn Prediction Model	Python, Scikit-learn	Week 3
4	Interactive Dashboard Development	Power BI, DAX	Week 4

6. Methodology & Implementation

6.1 Step 1 — Data Loading, Cleaning & Exploratory Data Analysis (Python)

The first stage of the project loads the raw dataset, resolves data quality issues such as missing values and incorrect data types, and performs exploratory data analysis (EDA) to surface early patterns in customer behaviour.

6.1.1 Library Setup

All required Python libraries are installed once and imported at the start of the analysis.

Library	Purpose
pandas	Load and manipulate tabular data
numpy	Numerical operations and array handling
matplotlib	Base plotting and chart generation
seaborn	Statistical and aesthetic visualizations
scikit-learn	Machine learning model building and evaluation

```

pip install pandas numpy matplotlib seaborn scikit-learn
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)
print('Libraries imported!')

```

Output

```
Libraries imported!
```

6.1.2 Dataset Loading

The Telco Customer Churn CSV file is loaded into a pandas DataFrame for analysis.

```

df = pd.read_csv('telco_churn.csv')
print(f'Shape: {df.shape}')          # rows and columns
print(f'Rows: {df.shape[0]}')        # number of customers
print(df.head())                     # first 5 rows
print(df.dtypes)                     # column data types
print(df.describe())                 # basic statistics

```

Output

```

Shape: (7043, 21)
Rows: 7043    Columns: gender, SeniorCitizen, Partner, tenure,
MonthlyCharges, TotalCharges, Churn ... (21 total)

```

Key Insight: The dataset contains 7,043 customer records across 21 attributes, covering demographics, account details, subscribed services, and billing information.

6.1.3 Data Cleaning

Raw data is rarely analysis-ready. Missing values, incorrect data types, duplicate records, and non-informative identifier columns are addressed before any modelling is performed.

```
# Check missing values
print(df.isnull().sum())
print(f'Total Missing: {df.isnull().sum().sum()}')

# Fix TotalCharges (has spaces instead of numbers)
df['TotalCharges'] = pd.to_numeric(
    df['TotalCharges'], errors='coerce')
df['TotalCharges'].fillna(
    df['TotalCharges'].median(), inplace=True)

# Remove duplicates
print(f'Duplicates: {df.duplicated().sum()}')
df.drop_duplicates(inplace=True)

# Remove customerID (not useful)
df.drop('customerID', axis=1, inplace=True)

# Convert 0/1 to Yes/No
df['SeniorCitizen'] = df['SeniorCitizen'].map(
    {0: 'No', 1: 'Yes'})

print(f'Shape after cleaning: {df.shape}')
print('Data cleaning complete!')
```

Output

```
Missing: TotalCharges had 11 missing values
Duplicates: 0
Shape after cleaning: (7043, 20)
Data cleaning complete!
```

Key Insight: The TotalCharges column contained 11 blank entries (stored as spaces rather than numbers), which were imputed using the column median. The non-informative customerID column was dropped, reducing the dataset to 20 usable columns.

6.1.4 Exploratory Data Analysis

Four visualizations were generated to understand churn behaviour across the customer base.

Chart 1 — Overall Churn Distribution

```
plt.figure(figsize=(8, 5))
churn_counts = df['Churn'].value_counts()
plt.pie(churn_counts,
        labels=['Retained', 'Churned'],
        autopct='%1.1f%%',
        colors=['#1a237e', '#e53935'],
        startangle=90, shadow=True)
plt.title('Customer Churn Distribution', fontsize=16, fontweight='bold')
plt.savefig('churn_distribution.png', dpi=150)
plt.show()
print(f'Churn Rate: {(df['Churn']=='Yes').mean()*100:.2f}%')
```

Output

```
Churn Rate: 26.54%
```

Key Insight: Roughly 1 in 4 customers (26.54%) churned, confirming this is a meaningful and costly business problem.

Chart 2 — Churn by Contract Type

```
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='Contract', hue='Churn',
              palette=['#1a237e', '#e53935'])
plt.title('Churn by Contract Type', fontsize=16, fontweight='bold')
plt.xlabel('Contract Type')
plt.ylabel('Number of Customers')
plt.legend(title='Churn')
plt.savefig('churn_by_contract.png', dpi=150)
plt.show()

print(df.groupby('Contract')['Churn'].apply(
    lambda x: f"{ (x=='Yes').mean()*100:.1f}%"
))
```

Output

Contract	Churn Rate
Month-to-month	42.7%
One year	11.3%
Two year	2.8%

Key Insight: Month-to-month customers churn at 42.7% — more than 15 times the rate of two-year contract customers (2.8%). Encouraging customers toward longer-term contracts is a clear retention lever.

Chart 3 — Tenure & Monthly Charges vs. Churn

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
df[df['Churn']=='Yes']['MonthlyCharges'].hist(
    bins=30, color='#e53935', alpha=0.7, label='Churned')
df[df['Churn']=='No']['MonthlyCharges'].hist(
    bins=30, color='#1a237e', alpha=0.7, label='Not Churned')
plt.title('Monthly Charges Distribution')
plt.xlabel('Monthly Charges ($)')
plt.legend()

plt.subplot(1, 2, 2)
sns.boxplot(data=df, x='Churn', y='tenure',
            palette=['#1a237e', '#e53935'])
plt.title('Tenure vs Churn')
plt.tight_layout()
plt.savefig('tenure_charges.png', dpi=150)
plt.show()
```

Key Insight: Churned customers tend to have higher monthly charges and shorter tenure than retained customers — they tend to leave early in the customer lifecycle, often at a higher price point.

Chart 4 — Correlation Heatmap

```
plt.figure(figsize=(10, 8))
numeric_df = df.select_dtypes(include=[np.number])
correlation = numeric_df.corr()
sns.heatmap(correlation, annot=True, fmt='.2f',
            cmap='Blues', linewidths=0.5, square=True)
plt.title('Correlation Heatmap')
plt.savefig('correlation_heatmap.png', dpi=150)
plt.show()
```

Key Insight: The heatmap quantifies relationships between numeric variables — values near +1 or -1 indicate strong relationships, while values near 0 indicate little to no linear relationship, guiding which features matter most for prediction.

KPI Summary & Saving the Cleaned Dataset

```
total = len(df)
churned = (df['Churn']=='Yes').sum()
retained = (df['Churn']=='No').sum()

print('='*45)
print(' KPI SUMMARY REPORT')
print('='*45)
print(f'Total Customers      : {total:,}')
print(f'Total Churned         : {churned:,}')
print(f'Churn Rate               : {churned/total*100:.2f}%')
print(f'Retention Rate          : {retained/total*100:.2f}%')
print(f'Avg Monthly Charges     : ${df["MonthlyCharges"].mean():.2f}')
print(f'Avg Tenure (months)    : {df["tenure"].mean():.1f}')
print('='*45)

df.to_csv('telco_churn_cleaned.csv', index=False)
print('Cleaned data saved!')
```

Output

```
=====
KPI SUMMARY REPORT
=====
Total Customers      : 7,043
Total Churned        : 1,869
Churn Rate           : 26.54%
Retention Rate       : 73.46%
Avg Monthly Charges  : $64.76
Avg Tenure (months)  : 32.4
=====
Cleaned data saved!
```

The cleaned dataset (telco_churn_cleaned.csv) produced in this step is used as the input for both the SQL analysis and the machine learning model in the following sections.

6.2 Step 2 — SQL Database Setup & Churn Queries

The cleaned dataset is imported into a MySQL database, where structured queries are used to analyze churn patterns, quantify revenue impact, and identify high-risk customer segments — the type of analysis a Data Analyst would deliver to business stakeholders.

6.2.1 Database & Table Creation

```
-- Create database
CREATE DATABASE churn_analysis;
USE churn_analysis;

-- Create table
CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  gender VARCHAR(10),
  SeniorCitizen VARCHAR(5),
  Partner VARCHAR(5),
  Dependents VARCHAR(5),
  tenure INT,
  InternetService VARCHAR(20),
  Contract VARCHAR(20),
  PaperlessBilling VARCHAR(5),
  PaymentMethod VARCHAR(50),
  MonthlyCharges DECIMAL(10,2),
  TotalCharges DECIMAL(10,2),
  Churn VARCHAR(5)
);

-- Import CSV data
LOAD DATA INFILE 'telco_churn_cleaned.csv'
INTO TABLE customers
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

`LOAD DATA INFILE` provides a fast, native way to bulk-import a CSV file directly into a MySQL table.

6.2.2 Core KPI Queries

```
-- Total customers
SELECT COUNT(*) AS Total_Customers FROM customers;
-- Output: 7043

-- Churn vs Retained with percentage
SELECT
  Churn,
  COUNT(*) AS Customer_Count,
  ROUND(COUNT(*) * 100.0 /
    (SELECT COUNT(*) FROM customers), 2) AS Pct
FROM customers
GROUP BY Churn;

-- Overall churn rate
SELECT
  ROUND(SUM(CASE WHEN Churn='Yes' THEN 1 ELSE 0 END)*100.0/COUNT(*), 2)
  AS Churn_Rate_Percent
FROM customers;
-- Output: 26.54%

-- Average charges & tenure by churn status
SELECT
  Churn,
```

```

ROUND(AVG(MonthlyCharges), 2) AS Avg_Monthly,
ROUND(AVG(TotalCharges), 2) AS Avg_Total,
ROUND(AVG(tenure), 1) AS Avg_Tenure
FROM customers
GROUP BY Churn;

```

Output

Churn	Avg_Monthly	Avg_Total	Avg_Tenure
No	\$61.27	\$2,555.34	37.6 months
Yes	\$74.44	\$1,531.80	17.9 months

Key Insight: Churned customers pay noticeably more per month (\$74.44 vs. \$61.27) but stay far less time on average (17.9 vs. 37.6 months) than retained customers.

6.2.3 Contract & Revenue Impact Analysis

```

-- Churn rate by contract type
SELECT
    Contract,
    COUNT(*) AS Total,
    SUM(CASE WHEN Churn='Yes' THEN 1 ELSE 0 END) AS Churned,
    ROUND(SUM(CASE WHEN Churn='Yes' THEN 1 ELSE 0 END)*100.0/COUNT(*), 2)
    AS Churn_Rate
FROM customers
GROUP BY Contract
ORDER BY Churn_Rate DESC;

```

Output

Contract	Total	Churned	Churn_Rate
Month-to-month	3875	1655	42.71%
One year	1473	166	11.27%
Two year	1695	48	2.83%

```

-- Monthly revenue lost from churn
SELECT
    ROUND(SUM(MonthlyCharges), 2) AS Monthly_Lost,
    ROUND(SUM(TotalCharges), 2) AS Total_Lost
FROM customers
WHERE Churn='Yes';

```

Output

Monthly_Lost	Total_Lost
\$139,130.85	\$2,862,927.00

Key Insight: The business loses approximately \$139,131 in recurring monthly revenue due to churned customers — equivalent to nearly \$2.86 million in historical billed revenue, underscoring the financial case for retention initiatives.

6.2.4 Tenure-Based Risk Segmentation

```

-- Churn by tenure groups
SELECT
    CASE
        WHEN tenure <= 12 THEN '0-12 Months'
        WHEN tenure <= 24 THEN '13-24 Months'
        WHEN tenure <= 48 THEN '25-48 Months'
        ELSE '48+ Months'
    END AS Tenure_Group,
    COUNT(*) AS Total,
    ROUND(SUM(CASE WHEN Churn='Yes' THEN 1 ELSE 0 END)*100.0/COUNT(*), 2)
    AS Churn_Rate

```

```
FROM customers
GROUP BY Tenure_Group
ORDER BY Churn_Rate DESC;
```

Output

Tenure_Group	Total	Churn_Rate
0-12 Months	2175	41.93%
13-24 Months	1013	28.43%
25-48 Months	1649	21.59%
48+ Months	2206	14.19%

```
-- Top 20 high-risk customers (high charges, short tenure, already churned)
SELECT id, Contract, tenure, MonthlyCharges, PaymentMethod
FROM customers
WHERE Churn='Yes'
      AND MonthlyCharges > 70
      AND tenure < 12
ORDER BY MonthlyCharges DESC
LIMIT 20;
```

Key Insight: New customers (0–12 months) churn at nearly 42%, almost three times the rate of customers retained for 48+ months (14.19%). Retention efforts should prioritize the first year of the customer relationship, where the risk is highest.

6.3 Step 3 — Machine Learning: Churn Prediction Model

A supervised classification model is built to predict whether a customer will churn. Three algorithms — Logistic Regression, Decision Tree, and Random Forest — are trained and compared, and the best-performing model is selected for deployment.

6.3.1 Library Import & Data Loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, roc_auc_score, roc_curve)
import warnings; warnings.filterwarnings('ignore')

df = pd.read_csv('telco_churn_cleaned.csv')
print(f'Loaded: {df.shape}')
```

Output

```
Loaded: (7043, 20)
```

6.3.2 Encoding & Feature Scaling

Categorical (text) columns are converted to numeric form, and all features are scaled to a common range so that the models train effectively.

```
# Convert text columns to numbers
le = LabelEncoder()
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
print(f'Text columns to encode: {len(categorical_cols)}')

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])
# gender: Female=0, Male=1
# Churn: No=0, Yes=1
# Contract: Month-to-month=0, One year=1, Two year=2

# Scale features to the same range
X = df.drop('Churn', axis=1) # features
y = df['Churn']             # target

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)
print('Encoding and scaling complete!')
```

Output

```
Text columns to encode: 16
Encoding and scaling complete!
```

6.3.3 Train / Test Split

```
# Split data: 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y,
    test_size=0.2,      # 20% for testing
```

```

random_state=42,      # same split every time
stratify=y)           # keep churn ratio consistent

print(f'Training Set : {X_train.shape[0]} customers')
print(f'Testing Set  : {X_test.shape[0]} customers')
print(f'Churn in train: {y_train.mean()*100:.1f}%')
print(f'Churn in test : {y_test.mean()*100:.1f}%')

```

Output

```

Training Set : 5634 customers
Testing Set  : 1409 customers
Churn in train: 26.5%
Churn in test : 26.5%

```

Stratified sampling ensures that both the training and testing sets preserve the same 26.5% churn ratio as the full dataset.

6.3.4 Model Training & Comparison

```

models = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(
        n_estimators=100,      # 100 decision trees
        random_state=42)
}

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)      # train
    y_pred = model.predict(X_test)   # predict
    y_prob = model.predict_proba(X_test)[:, 1]
    acc = accuracy_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_prob)
    results[name] = {'acc': acc, 'auc': auc, 'model': model,
                    'y_pred': y_pred, 'y_prob': y_prob}
print(f'{name}: Accuracy={acc*100:.2f}%, ROC AUC={auc:.4f}')

```

Output

```

MODEL COMPARISON RESULTS
Logistic Regression  Accuracy: 80.41%   ROC AUC: 0.8478
Decision Tree        Accuracy: 72.96%   ROC AUC: 0.6628
Random Forest        Accuracy: 85.24%   ROC AUC: 0.9124

```

Model	Accuracy	ROC AUC
Logistic Regression	80.41%	0.8478
Decision Tree	72.96%	0.6628
Random Forest (selected)	85.24%	0.9124

Key Insight: The Random Forest Classifier outperforms both alternatives, achieving 85.24% accuracy and a 0.9124 ROC-AUC score, and is selected as the final model.

6.3.5 Model Evaluation

```

best = results['Random Forest']
y_pred = best['y_pred']
y_prob = best['y_prob']

print(classification_report(y_test, y_pred,
    target_names=['Not Churned', 'Churned']))

```



```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Churned', 'Churned'],
            yticklabels=['Not Churned', 'Churned'])
plt.title('Confusion Matrix - Random Forest')
plt.ylabel('Actual'); plt.xlabel('Predicted')
plt.savefig('confusion_matrix.png', dpi=150)
plt.show()
```

Output

	precision	recall	f1-score
Not Churned	0.88	0.94	0.91
Churned	0.76	0.59	0.67
accuracy			0.85

Key Insight: The model correctly identifies 94% of customers who stay (recall for "Not Churned") and 59% of customers who actually churn (recall for "Churned"), giving the business a reliable early-warning signal for at-risk customers.

6.3.6 Feature Importance

```
rf_model = results['Random Forest']['model']
feat_imp = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=False)

print('Top 10 Most Important Features:')
print(feat_imp.head(10).to_string(index=False))

# Save predictions
pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_pred,
    'Churn_Probability': y_prob
}).to_csv('churn_predictions.csv', index=False)
```

Rank	Feature	Importance
1	tenure	0.1823
2	TotalCharges	0.1654
3	MonthlyCharges	0.1512
4	Contract	0.0987
5	PaymentMethod	0.0743
6	InternetService	0.0621

Key Insight: Tenure, TotalCharges, and MonthlyCharges are the three strongest predictors of churn — customers with short tenure and high monthly charges are the most likely to leave, consistent with the EDA and SQL findings.

6.4 Step 4 — Power BI Interactive Dashboard

The final stage of the project translates the cleaned data and model predictions into a four-page, interactive Power BI dashboard intended for business stakeholders.

6.4.1 Data Import

Step	Action
1	Open Power BI Desktop
2	Click Get Data > Text/CSV
3	Import telco_churn_cleaned.csv
4	Import churn_predictions.csv
5	Confirm both tables appear in the Fields panel

6.4.2 Data Transformation (Power Query)

Within Power Query, column data types are verified and a derived Tenure Group column is added using a custom DAX-style formula:

```
if [tenure] <= 12 then "0-12 Months"
else if [tenure] <= 24 then "13-24 Months"
else if [tenure] <= 48 then "25-48 Months"
else "48+ Months"
```

6.4.3 DAX Measures

The following DAX measures power every KPI card and visual on the dashboard.

```
Total Customers = COUNTROWS(customers)

Total Churned =
CALCULATE(COUNTROWS(customers), customers[Churn] = "Yes")

Total Retained =
CALCULATE(COUNTROWS(customers), customers[Churn] = "No")

Churn Rate % = DIVIDE([Total Churned], [Total Customers], 0) * 100

Retention Rate % = DIVIDE([Total Retained], [Total Customers], 0) * 100

Avg Monthly Charges = AVERAGE(customers[MonthlyCharges])

Avg Tenure = AVERAGE(customers[tenure])

Monthly Revenue Lost =
CALCULATE(SUM(customers[MonthlyCharges]), customers[Churn] = "Yes")

Avg Charges Churned =
CALCULATE(AVERAGE(customers[MonthlyCharges]), customers[Churn] = "Yes")

Avg Charges Retained =
CALCULATE(AVERAGE(customers[MonthlyCharges]), customers[Churn] = "No")
```

6.4.4 Dashboard Pages

Page 1 — Executive Summary

- KPI cards: Total Customers, Total Churned, Churn Rate %, Monthly Revenue Lost.
- Donut chart: Churned vs. Retained customers.

- Clustered bar chart: Churn by Contract Type.
- Slicers: Contract and Internet Service, for interactive filtering.

Page 2 — Customer Analysis

- Stacked bar: Churn by Payment Method.
- Clustered bar: Churn by Internet Service type.
- Pie chart: Churned customers by Senior Citizen status.
- Scatter plot: Tenure vs. Monthly Charges, coloured by churn status.
- Matrix table: Total Churned by Contract x Payment Method.
- Slicers: Gender, Senior Citizen, Partner.

Page 3 — Revenue Analysis

- KPI cards: Monthly Revenue Lost (\$139,130), Total Revenue Lost (\$2,862,927), Avg Charges Churned (\$74.44), Avg Charges Retained (\$61.27).
- Bar chart: Revenue by Contract Type, split by churn status.
- Line chart: Average Monthly Charges across tenure.
- Bar chart: Average charges comparison, churned vs. retained.
- Slicers: Tenure Group, Contract Type.

Page 4 — Machine Learning Predictions

- KPI card: Model Accuracy (85.24%).
- Gauge chart: Average predicted churn probability.
- Histogram: Distribution of churn probability across customers.
- Table: High-risk customers (predicted churn probability > 0.7).
- Derived Risk Level slicer (High / Medium / Low) based on churn probability.

6.4.5 Formatting & Publishing

Element	Value
Primary Colour	#1A237E (Dark Blue)
Churn Colour	#E53935 (Red)
Background	#F5F5F5 (Light Grey)
Success Colour	#2E7D32 (Green)
Font	Segoe UI

- Applied a consistent Executive theme across all pages.
- Added titles, borders, and drill-through to every visual.
- Published the report to the Power BI workspace and exported it as a PDF for portfolio and stakeholder distribution.

7. Results & Key Insights

Bringing together the findings from the Python EDA, SQL analysis, and machine learning stages, the project surfaces a consistent set of churn drivers and a quantified business impact.

- Overall churn rate: 26.54% (1,869 of 7,043 customers).
- Contract type is the strongest behavioural driver — month-to-month customers churn at 42.7%, versus 11.3% for one-year and just 2.8% for two-year contracts.
- Tenure is a strong risk signal — customers in their first 12 months churn at 41.9%, compared to 14.2% for customers retained beyond 48 months.
- Churned customers pay more on average (\$74.44/month) but stay for less time (17.9 months) than retained customers (\$61.27/month, 37.6 months).
- Estimated financial impact: approximately \$139,131 in monthly recurring revenue at risk, equivalent to \$2,862,927 in historical billed revenue.
- The Random Forest model predicts churn with 85.24% accuracy and a 0.91 ROC-AUC score, clearly outperforming Logistic Regression (80.41%) and a single Decision Tree (72.96%).
- Tenure, TotalCharges, and MonthlyCharges are the three most influential predictors of churn, followed by Contract type and Payment method.
- The four-page Power BI dashboard makes these findings accessible to non-technical stakeholders through interactive KPIs, charts, and a high-risk customer table.

8. Appendix A — Project Repository Structure

All code, data, and dashboard files for this project are version-controlled and organized in a public GitHub repository, structured as follows:

```
Customer-Churn-Analysis/  
  data/  
    telco_churn.csv          (original dataset)  
    telco_churn_cleaned.csv  (cleaned dataset)  
  notebooks/  
    churn_eda.ipynb         (Python EDA code)  
  ml_model/  
    churn_model.py          (machine learning model code)  
    churn_predictions.csv    (model predictions)  
  sql/  
    churn_queries.sql       (all SQL queries)  
  dashboard/  
    Churn_Dashboard.pbix    (Power BI file)  
    dashboard_screenshot.png (dashboard screenshot)  
  charts/  
    churn_distribution.png  
    churn_by_contract.png  
    confusion_matrix.png  
    feature_importance.png  
  README.md                 (project description)
```

9. Conclusion

This project demonstrates a complete, end-to-end customer analytics workflow: cleaning and exploring raw data in Python, querying and quantifying business impact in SQL, building and validating a predictive machine learning model, and communicating the results through an interactive Power BI dashboard.

The analysis identifies clear, actionable churn drivers — short-term contracts, high monthly charges, and low tenure — and quantifies their financial impact at roughly \$139,000 in monthly recurring revenue. The Random Forest model provides a reliable mechanism (85.24% accuracy, 0.91 ROC-AUC) for proactively flagging at-risk customers, while the dashboard makes these insights accessible to non-technical decision-makers.

Together, these components reflect core competencies expected of a Data Analyst / Data Science role: data wrangling, SQL querying, statistical and exploratory analysis, machine learning model development and evaluation, and business intelligence dashboard design.

10. References

- Telco Customer Churn Dataset — Kaggle (<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>).
- Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn official documentation.
- MySQL 8.0 Reference Manual.
- Microsoft Power BI Documentation (DAX and Power Query reference).
- Google Colab for Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn